

1. Usage of GenAI for Programming

- Most popular application for GenAI providers - with Claude (Anthropic) as most popular provider for programming <https://www.anthropic.com/research/anthropic-economic-index-september-2025-report> (Fig 3.1, 3.3 and 2.2)
 - Adoption of AI growing fast in enterprise (about 25% as of July 2025)
 - Most common application is software dev (36% of usage) and education (12%) next
 - High usage per capita in Israel
- Survey of existing practices [Gergely Orosz YouTube 25mins](#) July 2025 [Blog](#)
 - AI usage common: about 50% of developers use AI weekly
 - Who's using the technology: Mainly product managers and UX designers
 - Programmers as of 2025 save about 10% of their time using AI tools
 - Currently works better for individuals than teams (changing with Agent direction and CI/CD integration)
 - Adoption accelerating as tools have improved sufficiently to support daily usage
- How does AI change Software Dev?
 - Martin Fowler: similar to assembler to high-level programming (abstraction level increase) with introduction of *non-determinism*
 - Kent Beck: disruption similar to Microprocessors (1980s), Internet (2000s), Mobile (2010s) - change the landscape of "cheap vs expensive"
- [Use AI to improve](#) (Jeremy Howards and Chris Lattner, 30 Oct 2025) on software craftsmanship and AI - build software systems from basic principles

2. What the course is about

- Use LLMs (GenAI) to support software engineering
- Cover overall SDLC [Software Development Life Cycle](#)
 - Planning, Requirements, Design, Implementation, Testing, Deployment, Maintenance
 - Focus on how LLMs help for each stage
 - Address non-functional aspects (security, cost, responsible deployment)
- Assumes usage of CI/CD approach with DevOps tools
 - Adopt [GitHub CI/CD](#) platform with GitHub hosted projects and [GitHub Actions](#)
 - Integrate LLMs with environment through Tools and Agents (Claude Code, Gemini CLI) and AI-enabled IDEs (Cursor, VSCode).
- LLMs mechanisms
 - Base capabilities and usage (API, Prompting strategies)
 - Context engineering and Memory management
 - Tools invocation and MCP
 - Reasoning and Planning
 - Verification
 - We will focus on using the [DSPy framework](#)
- Project driven

- We will design and develop a software project to illustrate the topics learned
- We will attempt to build a micro-service distributed system
- Our project will use and expose AI capabilities and a RAG approach
- Attempt to reach observability and deployability using Docker

3. Logistics

- We'll use OpenRouter for LLM API access.
- We'll use LLMs mainly through tools that access API endpoints
 - CLI agents (Claude Clode, Gemini CLI, AllHands) and VSCode GitHub Copilot
- We'll use a single Github repo to evolve our shared project - coordination will be a challenge.
 - Organize in teams of up to 4 members each
 - Use Github tools to support communication:
 - Issues
 - PRs
 - Branches
- Weekly meetings will involve:
 - 1 hour presentation of course material
 - For each topic I will publish a list of relevant articles
 - You will pick
 - 1 hour review and planning of project evolution
 - We will aim for weekly iterations
 - Each team must commit new executable functionality every week
 - Integration tests will be critical
- Getting Started:
 - [Get started with GitHub Copilot in VS Code](#)
 - [Customize chat with instruction files](#)
 - [Get started with OpenHands](#)
 - [Get started with Gemini CLI](#)
 - [Get started with Claude Code](#)

4. Relevant Software Engineering Tasks

Example use cases:

- AllHands: [How we build OpenHands with OpenHands](#) (Graham Neubig, Aug 2025, 1 hour)
- [GitHub Copilot in VS Code](#) (Oct 2025)

Evaluation benchmarks: [SWE-Bench](#) - benchmark about fixing issues on Github.

We will elaborate on the various benchmarks and evaluation metrics after a review of the tasks.

4.1. 🎯 Requirement Engineering

This phase is about defining, documenting, and managing what the software should do. Agents can act as an assistant to a product manager or business analyst.

- **Clarifying Ambiguity:** An agent can scan a Product Requirements Document (PRD) or a set of user stories to **identify and flag ambiguous phrases** (e.g., "fast response time," "user-friendly interface"). It can then prompt the author with questions to help them define specific, testable acceptance criteria (e.g., "What is the acceptable P95 response time in milliseconds?").
- **Generating User Stories:** You can give an agent a high-level feature description (e.g., "Add a shopping cart to the e-commerce site"), and it can **break it down into a comprehensive list of user stories** and epics, covering common cases and edge cases (e.g., "As a shopper, I want to add an item," "As a shopper, I want to remove an item," "As a guest, I want to check out," etc.).
- **Identifying Conflicts:** When new requirements are added, a CLI agent can **analyze the entire requirement set for contradictions or conflicts** (e.g., "Requirement 3.1 states 'all data must be deleted after 24 hours,' but Requirement 5.2 introduces 'user history tracking.' Please clarify.").
- **Creating Behavioral Models:** An agent can take a set of requirements and **generate Gherkin (Given/When/Then) feature files**. These files can serve as both documentation and the foundation for Behavior-Driven Development (BDD) test suites.

4.2. 🎨 Software Design

This is about creating the low-level blueprint for components, modules, and interfaces.

- **API Specification Generation:** Given a set of requirements, an agent can **generate a complete OpenAPI (Swagger) specification** for a new REST API. This includes defining endpoints, request/response schemas, and standard HTTP error codes, which saves a massive amount of time.
- **Data Modeling:** You can describe your application's entities in natural language (e.g., "I need Users, Posts, and Comments. Posts have one User, but Users can have many Posts..."), and an agent can **generate the SQL schema (DDL)** or a NoSQL data model design.
- **Applying Design Patterns:** A developer can ask, "I need to notify multiple services when a user's profile is updated." An agent can **suggest the Observer pattern (or a Pub/Sub model)**, explain *why* it fits, and provide a code skeleton for its implementation in the project's language.
- **UI Prototyping:** A designer or PM can use a prompt to **generate prototype code for a UI component** (e.g., "Generate the React/CSS code for a dashboard card showing a user's name, profile picture, and online status dot.").

4.3. 🏛️ Software Architecture

This is the high-level structure of the system, its components, and how they interact. Agents can act as a "virtual" staff engineer to help you think through big-picture problems.

- **Generating Architecture Diagrams:** A CLI agent can be tasked to "scan the codebase, including infrastructure-as-code files (like Terraform or Docker Compose), and **generate a Diagrams as Code (e.g., PlantUML or Mermaid) script**." This script can then be rendered into a visual diagram showing the services, databases, and message queues.
- **Architectural Decision Records (ADRs):** After a team discussion, a tech lead can provide a summary to an agent (e.g., "We chose Kafka over RabbitMQ for the main event bus to handle high throughput and data re-playability"). The agent can then **format this into a formal ADR document**, complete with context, decision, and consequences.

- **Trade-off Analysis:** An architect can use an agent as a brainstorming partner. For example: "We need to design a highly available payment processing service. **Compare the pros and cons of a microservices approach versus a modular monolith** for this specific use case, focusing on fault isolation, scalability, and development complexity."
- **Non-Functional Requirement (NFR) Analysis:** An agent can "read this system design and **identify potential single points of failure**" or analyze it for other NFRs, such as scalability bottlenecks or security vulnerabilities (e.g., "The 'Auth Service' is a single point of failure. Consider deploying it in a high-availability (HA) configuration.").

4.4. 📝 Coding and Implementation

This is the most common application, but agents take it a step further than simple auto-complete.

- **Complex Code Generation:** Instead of just one line, an agent can be prompted to "scaffold the complete API boilerplate for a new user service, including CRUD endpoints, error handling, and basic validation."
- **Legacy Code Modernization:** A CLI agent can be tasked to "analyze this entire COBOL module, understand its business logic, and generate a functionally equivalent implementation in Python, complete with unit tests."
- **Large-Scale Refactoring:** This is a key area for CLI agents. You can issue commands like, "Perform a repo-wide refactor to replace the deprecated `X-Auth-Token` library with the new `Bearer-Token-Manager`, updating all function calls and import statements."
- **Dependency Management:** An agent can "scan the project for outdated or insecure dependencies, find the correct compatible versions, update the `package.json` or `requirements.txt`, and run the test suite to validate the changes."
- **Algorithm Optimization:** You can provide a function and ask an agent to "refactor this $O(n^2)$ search algorithm to a more efficient $O(n \log n)$ solution and explain the trade-offs."

4.5. 🧪 Testing and Quality Assurance

AI agents can automate the more cognitive aspects of testing, not just the execution.

- **Test Case Generation:** An agent can read a function or a user story (e.g., "As a user, I want to reset my password") and automatically generate a comprehensive suite of unit, integration, or end-to-end tests, including edge cases.
- **Intelligent Debugging:** Instead of just flagging a failed test, an agent can "run the failing test, analyze the stack trace and surrounding code, identify the probable cause of the bug, and propose a specific code fix."
- **Test Script Self-Healing:** When a UI element changes (e.g., a button ID), an agent can intelligently detect the broken test, find the new element, and automatically update the test script to fix it.
- **Code Review Automation:** An agent can be set up in a CI pipeline to act as a reviewer. It can "review this pull request, check for logic flaws, non-obvious bugs, and deviations from the project's architectural patterns," leaving actionable comments.

4.6. 📖 Documentation and Knowledge Management

GenAI excels at processing and generating natural language, making it perfect for documentation.

- **Automated Docstring Generation:** A CLI agent can be run across an entire codebase to "scan all public functions, analyze their logic and parameters, and generate (or update) comprehensive docstrings in the project's standard format."
- **README and Wiki Generation:** You can point an agent to a repository and ask it to "generate a `README.md` file that summarizes the project's purpose, lists its main features, and provides clear build and installation instructions."
- **Onboarding Assistance:** A new developer can ask an agent, "How do I set up the local development environment for the 'payments' service?" The agent can pull information from wikis, code, and other docs to provide a step-by-step guide.
- **Code Summarization:** When reviewing a large, unfamiliar file, a developer can ask the agent to "explain what this 500-line class does, what its primary responsibilities are, and how it interacts with other services."

4.7. DevOps and CI/CD

CLI agents are particularly powerful here, as they live in the terminal and can orchestrate complex workflows.

- **CI/CD Pipeline Generation:** A developer can describe their needs, and an agent can "generate the complete `gitlab-ci.yml` or GitHub Actions workflow file for this Node.js project, including build, test, lint, and deploy stages."
- **Automated Incident Response:** When an alert fires, an agent can be triggered to "start an incident triage: check the latest deployment, analyze the logs from the 'auth-service' pod in Kubernetes, and summarize any critical errors in the team's Slack channel."
- **Infrastructure as Code (IaC) Management:** You can prompt an agent to "write the Terraform script to provision a new auto-scaling EC2 instance group behind a load balancer, ensuring all security group rules are correctly configured."
- **Cost/Performance Optimization:** An agent could be tasked to "analyze our cloud spending for the last 30 days and suggest specific optimizations, like identifying unused EC2 instances or S3 buckets that could use a different storage tier."

5. Benchmarks and Evaluation

Benchmarks help us analyze how well current AI tools perform the tasks we aim for. The evaluation metrics used for each one are complex.

5.1. Coding & Implementation / Testing & Quality Assurance

These areas are the most mature in terms of standardized, automated benchmarks, often sharing the same underlying datasets.

Benchmark Name	Primary Task Type	Key Focus & Methodology
SWE-Bench	Implementation, Testing, Debugging	The gold standard. Models are challenged to generate code patches to resolve real-world GitHub issues (bugs/features) from open-source Python repositories. Success is measured by passing the official repository unit tests.

Benchmark Name	Primary Task Type	Key Focus & Methodology
SWE-Bench Pro	Implementation, Testing, Debugging	An actively maintained, more challenging variant of SWE-Bench. It focuses on long-horizon, enterprise-level problems that often require changes across multiple files, capturing greater real-world complexity.
HumanEval	Code Generation	Measures the ability to generate self-contained Python functions from docstrings. While still popular, it's considered less reflective of complex, real-world development than SWE-Bench.
MBPP (Mostly Basic Programming Problems)	Code Generation	Evaluates the generation of short Python programs from natural language descriptions, typically focusing on entry-level programming tasks.
CodeXGLUE	Code Repair, Defect Detection	A massive benchmark suite for various programming tasks, including specific subsets for identifying and fixing bugs (Code Repair/Defect Detection) , directly relevant to Quality Assurance.
LiveCodeBench / Aider Polyglot / xCodeEval	Implementation (Polyglot)	Focuses on multi-language code generation and modification (e.g., Python, C++, Go, JavaScript, Rust), providing a wider view of code competence beyond Python.
CLEVER	Code Generation	Formally verified code generation: maps a function signature + docstring in Python to Lean verification proofs.

5.2. Documentation & Knowledge Management

Evaluation in this category focuses on the model's ability to comprehend technical context and generate accurate, relevant explanations.

Benchmark Name	Primary Task Type	Key Focus & Methodology
CodeSummarization (from CodeXGLUE)	Documentation Generation	A specific task within CodeXGLUE that evaluates the model's ability to generate concise, meaningful natural language summaries for blocks of source code.

5.3. DevOps & CI/CD

This category is currently served by proxy benchmarks, as a unified "DevOps LLM Benchmark" is still emerging. The focus is on command execution and configuration generation.

Benchmark Name	Primary Task Type	Key Focus & Methodology
TerminalBench	Command Execution,	This benchmark focuses exclusively on the agent's ability to use a command-line interface (CLI) to complete

Benchmark Name	Primary Task Type	Key Focus & Methodology
	Automation	software tasks. This is a crucial skill for executing build scripts, configuring environments, and interacting with Git in a CI/CD context.
SWE-Lancer	Autonomous Task Completion	Although less actively reported on, this benchmark maps agent work to economic value (like Upwork tasks). The success often relies on the agent's ability to perform end-to-end CI/CD-like steps (e.g., setting up the environment, running tests, creating a PR).
DORA Metrics (as an evaluation framework)	Automation Performance	While not a dataset, DORA metrics (Deployment Frequency, Lead Time for Changes, etc.) are the de-facto industry standard for measuring DevOps performance. AI models are often evaluated on their ability to <i>improve</i> these metrics (e.g., auto-generating build scripts to reduce Lead Time).

5.4. Requirement Engineering, Design & Architecture

These highly cognitive tasks currently rely on more specialized, domain-specific benchmarks that test reasoning and trade-off analysis.

Benchmark Name	Primary Task Type	Key Focus & Methodology
DevBench	Design	Includes a benchmark on PRD to UML architecture design specifications, and environment setup (PRD+UML to Dependency Files).

The scoring of Large Language Models (LLMs) in software engineering benchmarks is divided into three main categories, each employing specialized metrics to assess **functional correctness**, **text quality**, and **real-world impact**.

6. Evaluation Metrics

6.1. Coding and Testing Metrics

This category, including benchmarks like **SWE-Bench**, **HumanEval**, and **MBPP**, focuses on the ability to generate syntactically correct and functionally correct code.

Metric	Definition	Benchmark Context
Pass@ <i>k</i>	The probability that at least one of <i>k</i> generated solutions for a problem passes the provided unit tests.	Standard for HumanEval and MBPP (single-function code generation). It measures <i>potential</i> ability.
% Resolved	The percentage of real-world GitHub issues (bugs or features) where the	Primary metric for SWE-Bench (end-to-end software development agent tasks).

Metric	Definition	Benchmark Context
	model's generated code patch passes all relevant unit and integration tests.	It measures a complete, successful fix/implementation.
Functional Correctness	A binary score (Pass/Fail) for a given code sample based on its unit test results.	The underlying principle for all code generation benchmarks. The code must execute correctly against the specified input/output examples.

6.2. Documentation and Design Metrics

This category, covering tasks like code summarization (**DocCodeEval**) and design/requirements question-answering (**DesignQA**), measures the model's ability to generate human-quality, relevant, and factual text.

A. Overlap/Similarity Metrics (Computation-Based)

These compare the model's generated output against a human-written "reference" or "ground truth."

- **ROUGE (Recall-Oriented Understudy for Gisting Evaluation):** Commonly used for summarization tasks. It measures the overlap of n -grams (sequences of words) between the generated text and the reference text. A higher score means the generated text covers more of the key information in the reference.
- **BLEU (Bilingual Evaluation Understudy):** Originally for machine translation, it is also used for code summarization. It measures the precision of n -grams in the generated text against the reference, often applying a **brevity penalty** if the output is too short.
- **Exact Match:** The simplest metric, which counts the percentage of answers that are **verbatim identical** to the ground truth. This is most relevant for short, factual questions.

B. Quality/Rubric Metrics (LLM-as-a-Judge)

For complex design or documentation tasks, a separate LLM (or a human) is often used to evaluate the output against specific criteria, known as **rubrics**.

- **Instruction Following:** Does the generated text adhere to all constraints (e.g., "must be less than 500 words," "must include a diagram reference")?
- **Grounding / Factuality:** Is the information in the response factually consistent with the provided source text (e.g., the design document or source code)?
- **Relevance / Coherence:** Is the answer directly relevant to the prompt, and is it logically structured and well-written?

6.3. DevOps and Architecture Metrics

Evaluation in the DevOps domain often uses the industry-standard **DORA (DevOps Research and Assessment) Metrics** to measure the impact of an AI assistant on the *speed* and *stability* of the

software delivery process. For terminal execution benchmarks (**TerminalBench**), task-level metrics are used.

A. DORA Metrics (High-Level Outcomes)

The Four Key DORA metrics are divided into **Throughput** (Speed) and **Stability**.

Category	Metric	Goal
Throughput	Deployment Frequency	Increase: How often an organization successfully releases to production.
Throughput	Lead Time for Changes	Decrease: Time from code commit to successful deployment in production.
Stability	Change Fail Percentage	Decrease: Percentage of deployments causing a failure in production.
Stability	Time to Restore Service (MTTR)	Decrease: Time it takes to recover from a failure in production.

B. Task-Level Metrics

- **Task Completion Rate:** The percentage of discrete tasks (e.g., configuring a server, running a CI/CD pipeline, executing a bash command) that the model or agent successfully completes.
- **Latency / Execution Time:** The time taken for the AI to complete the full sequence of actions required for the task.